5. ROS2 Function Package

1. Introduction to Function Packages

Each robot may have many functions, such as motion control, visual perception, autonomous navigation and so on. Of course, it is possible to put the source code of all these functions together, but when we want to share some of the functions with others, we will find that the code is all mixed together and is hard to split out.

Function packages work on this principle: we divide the code of different functions into different function packages and try to reduce the coupling between them as much as possible. When we need to share a package with others in the ROS community, we only need to explain how to use this function package, and others will be able to use it very quickly.

Therefore, the function package mechanism is one of the important ways to improve the software reuse rate in ROS.

2. Creating a Function Package

How do you create a function package in ROS2? We can use this command:

```
ros2 pkg create <package_name> --build-type <build-type> --dependencies <dependencies> --
node-name <node-name>
```

In the ros2 command:

- **pkg**: indicates the function package related functionality;
- **create**: indicates creating a function package;
- **package_name**: required: the name of the new function package;
- **build-type**: required: indicates whether the newly created function package is C++ or Python. If C++ or C is used, put ament_cmake here; if Python is used, put ament_python;
- **dependencies**: optional: indicates the dependencies of the function package. A C++ function package must include rclcpp, a Python function package must include rclpy, as well as any other dependencies needed;
- **node-name**: optional: the name of the executable program; the corresponding source file and configuration files are generated automatically;

For example, create the C++ and Python versions of the function package in the terminal:

- Switch to the src directory of the workspace
- Replace workspace here with your own actual folder path

```
cd workspace/src
```

- Example of creating a C++ function package

```
ros2 pkg create pkg_helloworld_cpp --build-type ament_cmake --dependencies rclcpp --node-
name helloworld
```

- Example of creating a Python function package

```
ros2 pkg create pkg_helloworld_py --build-type ament_python --dependencies rclpy --node-name
helloworld
```

3. Building the Function Package

In the function package that has been created, we can continue writing the code. Afterwards it needs to be built and the environment variables need to be configured before it can run normally:

- Switch to the workspace directory
- Build all the function packages

```
colcon build
```

- Build the specified function packages

```
colcon build --packages-select pkg1 pkg2
```

4. The Complete Workspace Structure with Function Packages

The directory structure of a ROS2 workspace is as follows:

```
WorkSpace --- the custom workspace.
    |--- build: the directory that stores the intermediate files; a separate subdirectory is
created for each function package under it.
    |--- install: the installation directory; a separate subdirectory is created for each
function package under it.
    |--- log: the log directory, used to store the log files.
    |--- src: the directory used to store the source code of the function packages.
        |-- C++ function package
            |-- package.xml: package information, such as the package name, version, author
and dependencies.
            |-- CMakeLists.txt: the build rules, such as the source files, dependencies and
target files.
            |-- src: the C++ source file directory.
            |-- include: the header file directory.
            |-- msg: the message interface file directory.
            |-- srv: the service interface file directory.
            |-- action: the action interface file directory.
        |-- Python function package
            |-- package.xml: package information, such as the package name, version, author
and dependencies.
            |-- setup.py: similar to the CMakeLists.txt of a C++ function package.
            |-- setup.cfg: the basic configuration file of the function package.
            |-- resource: the resource directory.
            |-- test: stores the test related files.
```

```
            |-- the directory with the same name as the package: the Python source file
directory.
```

In addition, whether it is a Python function package or a C++ function package, you can customize some directories related to configuration files.

```
|-- C++ or Python function package
    |-- launch: stores the launch files.
    |-- rviz: stores the rviz2 configuration related files.
    |-- urdf: stores the robot modeling files.
    |-- params: stores the parameter files.
    |-- world: stores the simulation environment related files.
    |-- map: stores the map files required for navigation.
    |-- ......
```

The above directories can also be given other names, or you can create other directories as needed.